

Material complementar - Core Data

1. Cláusula WHERE (Filtragem Simples) No Core Data, o WHERE é implementado através do NSPredicate.

// Filtra apenas pessoas que moram em "São Paulo"

```
@FetchRequest(  
    sortDescriptors: [],  
    predicate: NSPredicate(format: "city == %@", "São Paulo")  
)  
var persons: FetchedResults<Person>
```

2. Cláusula LIKE (Busca por Texto/Busca Parcial) Para simular o LIKE %texto%, usamos o operador CONTAINS. Adicionamos [cd] para ignorar maiúsculas/minúsculas (case-insensitive) e acentos (diacritic-insensitive).

// Busca pessoas cujo nome contém a letra ou palavra "ana" (ex: Ana, Mariana)

```
@FetchRequest(  
    sortDescriptors: [],  
    predicate: NSPredicate(format: "name CONTAINS[cd] %@", "ana")  
)  
var persons: FetchedResults<Person>
```

Outras variações úteis:

- Começa com (Swift equivalente a LIKE 'text%'): NSPredicate(format: "name BEGINSWITH[cd] %@", "A")
- Termina com (Swift equivalente a LIKE '%text'): NSPredicate(format: "name ENDSWITH[cd] %@", "silva")

3. Cláusula ORDER BY (Ordenação) A ordenação é feita utilizando a classe NSSortDescriptor. Você pode passar múltiplos critérios ordenados por prioridade.

// Ordena por Cidade (Crescente) e depois por Nome (Decrescente)

```
@FetchRequest(  
    sortDescriptors: [  
        NSSortDescriptor(keyPath: \Person.city, ascending: true),  
        NSSortDescriptor(keyPath: \Person.name, ascending: false)  
    ]  
)  
var persons: FetchedResults<Person>
```

4. Combinando Tudo (Exemplo Avançado) É possível misturar ordenação (ORDER BY) e filtragem (WHERE / LIKE) no mesmo bloco de código.

// Busca pessoas de "Curitiba" QUE tenham "Silva" no nome, ordenadas por Nome

```
@FetchRequest(  
    sortDescriptors: [NSSortDescriptor(keyPath: \Person.name, ascending: true)],  
    predicate: NSPredicate(format: "city == %@ AND name CONTAINS[cd] %@",  
        "Curitiba", "Silva")  
)  
var filteredPersons: FetchedResults<Person>
```

5. Cláusulas GROUP BY e Funções de Agregação O Core Data não possui uma cláusula GROUP BY direta no @FetchRequest. Para agrupar dados (ex: contar quantas pessoas existem por cidade), precisamos configurar uma NSFetchRequest manual utilizando NSDictionaryResultType.

```
func countPersonsByCity() {
    let request = NSFetchRequest<NSFetchDictionaryResultType>(entityName: "Person")
    request.resultType = .dictionaryResultType

    // 1. Definir a propriedade de agrupamento (GROUP BY city)
    request.propertiesToGroupBy = ["city"]

    // 2. Criar a expressão de contagem (COUNT(id))
    let countExpression = NSEExpressionDescription()
    countExpression.name = "totalPersons"
    countExpression.expression = NSEExpression(forFunction: "count:", arguments:
[NSEExpression(forKeyPath: "id")])
    countExpression.expressionResultType = .integer32AttributeType

    // 3. Definir o que a query deve retornar (A cidade e a contagem)
    request.propertiesToFetch = ["city", countExpression]

    // 4. Executar a busca
    if let results = try? viewContext.fetch(request) {
        for result in results {
            let city = result["city"] as? String ?? "Desconhecida"
            let total = result["totalPersons"] as? Int ?? 0
            print("Cidade: \(city) -> Total de pessoas: \(total)")
        }
    }
}
```

Links úteis:

- <https://developer.apple.com/documentation/coredata>
- <https://www.hackingwithswift.com/quick-start/swiftui/introduction-to-using-core-data-with-swiftui>
- <https://www.kodeco.com/7569-getting-started-with-core-data-tutorial>